



Runbook and Support Guide — TASCO Growth Platform

TASCO Insurance × VETC Growth Platform — Operations

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

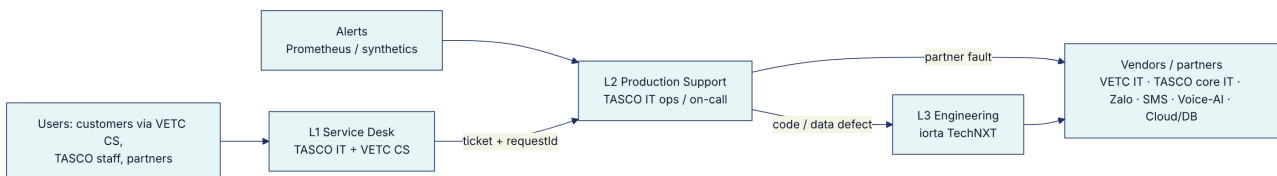
1. Support model
2. System quick reference
3. Incident runbooks
4. Standard operating procedures
5. Health-check routine (daily, L2)
6. L1 known-error quick answers (KEDB)
7. Incident communication
8. Escalation contacts

Document	Version	Date	Owner	Status
Runbook and Support Guide	1.0	07 October 2026	TASCO IT Production Support (L2), with iorta TechNXT engineering (L3)	Issued for submission

Item	Value
Service	tasco-growth-platform (log field service), Node.js 22, PostgreSQL 16
Owner (BAU)	TASCO IT Production Support (L2), with iorta TechNXT engineering (L3) during hypercare and under the AMS contract
Environments	Production (Kubernetes, deploy/k8s); hosted UAT on Railway at https://tasco-growth-api-uat.up.railway.app (NODE_ENV=uat, PostgreSQL, sandbox adapters, demo accounts)
Related	Monitoring and Alerting · DR and BCP · Release and Change Management · Known issues KI-xx: Test Strategy, Appendix A (docs/quality/test-strategy.pdf)
Review	Applies to the pilot (G3). Reviewed after each hypercare week

Sandbox and production. Today the integration adapters in the repo are sandboxes (`mockGateways.js` , `simulatedCaller.js`). The procedures below are written for production adapters with the **same names**: `vetc-wallet` , `tasco-core` , `voice-ai` , `app-push` , `zalo-zns` and `sms` . Partner-side contacts and portals are placeholders until the interface agreements (IIAs) are signed.

1. Support model



Tier	Who	Hours	Scope	Tools / access
L1 Service Desk	TASCO IT service desk (staff and partner issues); VETC customer service (customers)	08:00–20:00 ICT, 7 days; out of hours via the on-call phone for Sev 1	Intake, triage with the severity matrix, known-error answers (KEDB), password-change guidance, capture of X-Request-Id / time / user / route	ITSM tool, this guide §6 (FAQ), read-only dashboards
L2 Production Support	TASCO IT production support engineers (support_engineer role)	24×7 on-call rota	Runbooks RB-xx and SOP-xx, job reruns, breaker and backlog checks, config changes through CAB, escalation	GET /api/ops/status, GET /api/ops/jobs, POST /api/ops/jobs/:kind, Grafana, log search, kubectl (namespace-scoped), read-only DB replica
L3 Engineering	iorta TechNXT engineers (hypercare, then the AMS contract)	Business hours; 24×7 on-call for Sev 1 during hypercare	Code and data defects, hotfixes, DB data fixes under change control, root-cause analysis (RCA)	Repo, CI/CD, DB via break-glass with approval

Tier	Who	Hours	Scope	Tools / access
Vendors / partners	VETC IT (wallet, SSO, app, events), TASCO core IT (policy admin), Zalo OA/ZNS, SMS brandname provider, voice-AI vendor, cloud/DB provider	Per contract	Faults in their systems	Their portals and status pages

1.1 Severity matrix and SLAs

Severity	Definition	Examples	Response (ack)	Update cadence	Restore / workaround	Resolve (permanent fix)
Sev 1 — Critical	Production down or core revenue path broken for many users; security or privacy breach; regulatory breach in progress	/health/ready failing on all pods; purchases failing > 20 %; customers charged without cover at scale; messages with banned wording sent; contact to DNC customers; audit chain verification fails; PII exposure	15 min (24×7)	Every 30 min	4 h	5 business days (RCA within 5 business days)
Sev 2 — High	Major function degraded; workaround exists; single partner/channel down	Breaker open on zaLo-zns; journeys not run for the day; partner API down for one partner; dead-letter events growing; telesales queue unavailable in one region	30 min (24×7)	Every 2 h	8 h	10 business days
Sev 3 — Medium	Minor function impaired; low business impact	Dashboard slow; a single customer cannot verify a certificate; one user locked out	4 business h	Daily	3 business days	Next release
Sev 4 — Low	Cosmetic, question, or request	Label wording, report request	1 business day	On change	10 business days	Backlog

SLA clock: from alert or ticket creation. A Sev 1 or Sev 2 **opens a war room** (Teams/Zalo group "TASCO-GP-INCIDENT") with an Incident Commander (L2 lead), a Communications Lead and the L3 engineer.

1.2 On-call

Item	Arrangement
Rota	Weekly, Monday 09:00 ICT handover; primary + secondary L2; L3 secondary from iorta during hypercare (4 weeks after each wave)
Paging	Alertmanager → PagerDuty/Opsgenie (or the TASCO equivalent) for severity: <code>page</code> ; Teams/Zalo channel for severity: <code>ticket</code>
Handover	Open incidents, alerts silenced (with expiry), pending changes, jobs to watch (journeys at 08:15 ICT, reconcile nightly at 02:00 ICT)
Escalation	Primary (15 min) → secondary (15 min) → L2 lead → TASCO IT Head (Sev 1 > 1 h)
Tools checklist	VPN, <code>kubectl</code> context, Grafana, log search, ITSM, this runbook, partner contact list (§9)

2. System quick reference

Item	Value
Probes	<code>GET /health/live</code> (process up), <code>GET /health/ready</code> → {status, store, db}; 503 when the DB ping fails, during shutdown, or before the server listens
Metrics	<code>GET /metrics</code> (Prometheus text; bearer <code>METRICS_TOKEN</code> when set). Names in Monitoring and Alerting §3
Ops status	<code>GET /api/ops/status</code> (perm ops:read): store, integrations[{name, circuit: closed open half_open}], rules[{kind, version, checksum}], eventBacklog{pending, processing, done, dead_letter}, auditEntries
Job history	<code>GET /api/ops/jobs</code> : recent runs with kind, status, result, error (recorded for reconciliation and retention)
Run a job (API)	<code>POST /api/ops/jobs/reconciliation, .../retention, .../relay</code> (perm ops:run_jobs)
Run a job (CLI)	<code>npm run job -- <migrate seed rules journeys recompute reconcile retention relay openapi></code> (src/jobs/cli.js). seed refuses to run unless <code>DEMO_MODE=true</code> and is never used in production
Scheduled jobs (CronJobs, <code>deploy/k8s/60-jobs.yaml</code> , times in UTC)	tasco-growth-journeys daily 01:15 UTC = 08:15 ICT ; tasco-growth-recompute nightly 17:30 UTC (00:30 ICT); tasco-growth-reconcile nightly 19:00 UTC (02:00 ICT); tasco-growth-retention nightly 20:00 UTC (03:00 ICT); tasco-growth-relay every 5 min as a safety net (every API pod also relays every 1 s). All use <code>concurrencyPolicy: Forbid</code> . The tasco-growth-migrate Job runs before each rollout
Graceful shutdown	<code>SIGTERM</code> → readiness false → stop relay → close server → exit 0; forced exit after 25 s. The pod's <code>terminationGracePeriodSeconds</code> must be ≥ 30
Circuit breakers	5 consecutive failed calls (after 2 retries each, 100 ms jittered backoff, 5 s timeout) → open for 30 s → half-open probe. voice-ai: 30 s timeout, no retries
Outbox	domain_events statuses pending → processing → done; on handler failure back to pending; after 5 attempts dead_letter

Item	Value
Key config (env)	DATABASE_URL, DB_POOL_MAX (10), JWT_SECRET, JWT_TTL_SECONDS (1800), DATA_KEYS, DATA_KEY_ACTIVE, BLIND_INDEX_KEY, MFA_REQUIRED_ROLES, CORS_ORIGINS, RATE_LIMIT_*, LOCKOUT_*, BODY_LIMIT_BYTES, TRUST_PROXY, DEMO_MODE (must be false), MIGRATE_ON_START (set false in Kubernetes; the migration Job runs first), METRICS_TOKEN (bearer for /metrics), LINK_TTL_DAYS (30), TRUST_PROXY_HOPS (1, the right-most trusted hop of X-Forwarded-For). Secrets also via <NAME>_FILE. UAT only: DEMO_PASSWORD, DEMO_TOTP_SEED, DEMO_ACCOUNT_SYNC (never set in production)

2.1 Logs

The service logs one JSON object per line to stdout. PII keys are redacted ([REDACTED]).

Log	Level	Fields	Meaning
request	info	requestId, method, route (e.g. POST /api/customer/orders), status, ms	Every API request (not health, metrics or static)
unhandled error	error	requestId, path, err.message, err.stack	500 INTERNAL_ERROR: always investigate
circuit opened / circuit closed	warn / info	integration, failures	Breaker transitions
event handler failed	error	type, handler, eventId, err	Outbox subscriber failure (retried)
outbox relay failed	error	err	Relay loop error (usually the DB)
job failed / job finished	error / info	kind or job, result	Jobs
journey run complete	info	date, due, done, skipped, cancelled, deferred, byChannel, byJourney	Daily journeys
order completed	info	orderId, channel	Sale
batch ingested	info	batchId, records, profiles	Ingestion
rule set activated	info	kind, id	Rule change live
pg pool error	error	err	DB connectivity
shutting down / server listening / migrations applied	info	signal / port, store / applied	Lifecycle
fatal startup error	(stderr)	err	Boot failure (missing secret or DATABASE_URL in production, DEMO_MODE=true in production, bad DATA_KEYS, DB unreachable)

Log queries (Loki LogQL shown; the `jq` equivalent works on `kubectrl logs`):

```
# All 5xx in the last 15 min by route
sum by (route) (count_over_time({app="tasco-growth-api"} | json | msg="request" | status ≥ 500
[15m]))

# Trace one request end to end
{app="tasco-growth-api"} | json | requestId="3f2a9c1e-..."

# Slow purchases (> 3 s)
{app="tasco-growth-api"} | json | msg="request" | route="POST /api/customer/orders" | ms > 3000

# Breaker transitions
{app="tasco-growth-api"} | json | msg=~"circuit (opened|closed)"

# Failed sign-ins (401 on login)
sum(count_over_time({app="tasco-growth-api"} | json | route="POST /api/auth/login" | status="401"
[5m]))
```

```
kubectl -n tasco-growth logs deploy/tasco-growth-api --since=15m | jq -c
'select(.msg=="request" and .status≥500) | {ts,requestId,route,status,ms}'
kubectl -n tasco-growth logs deploy/tasco-growth-api --since=1h | jq -c
'select(.level=="error") | {ts,msg,requestId,err:.err.message}'
```

The **audit trail** (DB, hash-chained) holds security and business events such as `auth.login_failed`, `auth.login_locked`, `auth.mfa_failed`, `auth.mfa_enrolled`, `user.reset`, `rules.approved`, `quote.sent_to_customer`, `order.issuance_failed`, `consent.withdrawn` and `dsar.erased`. Query it with `GET /api/audit?action=<action>&actor=<id>&entityId=<id>` (perm `audit:read`).

3. Incident runbooks

Each runbook has **Detect** → **Diagnose** → **Remediate** → **Verify** → **Follow-up**. Always record the `requestId` (s), timestamps (ICT) and actions in the incident ticket.

RB-01 — Database down / readiness failing

Detect	ALR-01 ReadinessFailing, ALR-02 HighErrorRate; /health/ready → 503 {db:false}; logs pg pool error, outbox relay failed; Kubernetes removes pods from Service endpoints
Severity	Sev 1 when all pods are not ready
Diagnose	1. <code>kubectl -n tasco-growth get pods -l app=tasco-growth-api (Ready 0/1?)</code> . 2. <code>curl -s https://<host>/health/ready</code> → check store is postgres. Production refuses to start without <code>DATABASE_URL</code> (KI-28 fixed), so memory means <code>ALLOW_MEMORY_STORE</code> was set or the pod is not running with <code>NODE_ENV=production</code> : treat as Sev 1 misconfiguration. 3. DB provider console: instance status, failover events, storage full, CPU, connections. 4. <code>SELECT count(*), state FROM pg_stat_activity GROUP BY 2;</code> (connection exhaustion: pods × <code>DB_POOL_MAX</code>). 5. Check for expired DB credentials or TLS certificates (<code>DATABASE_SSL</code> defaults on in production with <code>rejectUnauthorized:true</code>)
Remediate	Failover in progress: wait (managed HA: 60–120 s); the app reconnects by itself, no restart needed. Storage full: extend storage; then check for runaway tables (<code>domain_events</code> , <code>audit_log</code> , KI-13). Connections exhausted: scale the deployment down to the HPA minimum, restart the pods holding idle-in-transaction connections, enable or resize PgBouncer. Credentials/TLS: rotate the secret (<code>DATABASE_URL_FILE</code>), rolling restart. Primary lost and not recoverable: invoke DR procedure FO-1 (DR and BCP §6)
Verify	/health/ready 200 on every pod; GET /api/ops/status responds; eventBacklog.pending draining; run POST /api/ops/jobs/reconciliation for the outage window

Follow-up	Reconcile orders created during the outage (RB-11); run any missed journeys (RB-07) inside the contact window
------------------	---

RB-02 — Circuit open: VETC wallet (vetc-wallet)

Detect	ALR-10 <code>CircuitOpen{integration="vetc-wallet"}; /api/ops/status integration vetc-wallet circuit:"open";</code> customers get 503 UPSTREAM_UNAVAILABLE at payment; orders <code>payment_failed</code>
Severity	Sev 1 if it lasts > 15 min in business hours (revenue path); otherwise Sev 2
Diagnose	1. sum by (result) (rate(integration_calls_total{integration="vetc-wallet"}[5m])): all errors, or timeouts? 2. Logs: <code>circuit opened</code> , <code>unhandled error</code> , upstream messages ("timed out after 5000ms"). 3. VETC status page / VETC IT bridge. 4. Network: egress allow-list / NetworkPolicy, DNS, mTLS certificate expiry
Remediate	The breaker is self-healing : every 30 s one request probes (half-open). Do not restart pods to "reset" it, because that only causes more load on VETC. 1. Raise a Sev 1/2 with VETC IT. 2. Ask VETC CS to post the in-app notice "Thanh toán tạm gián đoạn" if > 15 min. 3. Pause the journey CronJob if the outage is during the 08:15 run, so reminders do not drive customers into a broken payment flow (SOP-07). 4. Telesales: continue quoting and sending quotes (<code>POST /api/quotes/:id/send</code>); customers pay once the wallet recovers. Staff cannot take payment (enforced in code)
Verify	Circuit closed; <code>orders_completed_total</code> resumes; error ratio < 1 %
Follow-up	Debits that timed out on our side may have been captured by VETC (KI-18) : list <code>payment_failed</code> orders in the window (RB-11 query) and reconcile them with the VETC settlement file; refund or issue as appropriate

RB-03 — Circuit open: TASCO core (tasco-core)

Detect	ALR-10 for <code>tasco-core</code> ; ALR-84; orders <code>issuance_failed_refunded</code> or <code>compensation_failed</code> ; <code>orders_compensation_total</code> ; audit <code>order.issuance_failed</code>
Severity	Sev 1: customers are charged and then refunded. Saga compensation cancels issued lines and refunds in full; any <code>compensation_failed</code> order needs manual action (RB-11)
Diagnose	As RB-02 for <code>integration="tasco-core"</code> ; contact TASCO core IT (policy admin)
Remediate	1. Pause sales entry points if the outage will last more than 30 min: suspend the journeys CronJob (SOP-07) and ask VETC to hide the renew banner (app config); partners are informed via the partner status channel. 2. Each failed order follows RB-11
Verify	Circuit closed; a test issuance in production with an internal test vehicle (pre-agreed with TASCO core), then cancel it

RB-04 — Circuit open: Zalo ZNS (zalo-zns), SMS (sms) or app push (app-push)

Detect	ALR-10; ALR-31 <code>MessageFailureRatio</code> ; <code>messages_total{status="failed",channel=...}</code>
Severity	Sev 2 (journeys degrade; customers can still self-serve)
Diagnose	Zalo: OA token expiry, template disabled or rejected by Zalo, daily quota exceeded. SMS: brandname registration, provider balance. Push: credentials and certificates (FCM/APNs)
Remediate	Touchpoints automatically fall back to the next channel in their step (e.g. [<code>app_push</code> , <code>zalo_zns</code> , <code>sms</code>]). If all channels in a step fail, the touchpoint is skipped (terminal). For long outages, suspend the journeys CronJob (SOP-07) and resume after recovery, then re-queue the skipped touchpoints (RB-07)
Verify	<code>messages_total{status="sent"}</code> resumes for the channel

RB-05 — Circuit open or failures: voice-AI (voice-ai)

Detect	ALR-10 for <code>voice-ai</code> ; <code>voice_calls_total</code> flat in business hours; journeys <code>voice_bot</code> steps failing
---------------	---

Severity	Sev 2. BCP: DR and BCP §8
Remediate	Vendor bridge. Pause voice campaigns (POST /api/voice/campaign is manual, so stop launching it). Journeys: the voice_bot steps fail; telesales steps continue. Supervisors raise agent capacity for hot leads from GET /api/leads?tier=hot

RB-06 — Outbox backlog or dead-letter events

Detect	ALR-20 DeadLetterEvents (increase(events_processed_total{status="dead_letter"}[15m]) > 0); ALR-21 OutboxBacklog; GET /api/ops/status → eventBacklog
Severity	Sev 2 (side effects such as lead recompute, confirmation messages and cross-sell scheduling are delayed or lost)
Diagnose	1. GET /api/ops/status → eventBacklog e.g. {"done": 812345, "pending": 15000, "processing": 37, "dead_letter": 12}. 2. Logs: event handler failed with type, handler, eventId, err. 3. Inspect dead letters on the read-only replica (query Q1 below). 4. Stuck processing (a replica died mid-relay): the Postgres relay re-claims these automatically after 5 minutes . If any are older than 10 minutes, the relay itself is not running (all pods failing outbox relay failed); count them with query Q2 below. 5. Large pending with no dead letters: the relay is slow (DB, slow handler such as a full recompute), or every pod is failing to relay (outbox relay failed)
Remediate	Fix the cause first (bad rule set, missing profile, DB issue). Then, under a change ticket (L3 + DBA), re-queue . There is no API for this yet (KI-03): use queries Q3 (stuck processing) and Q4 (selected dead letters) below. Then drain: POST /api/ops/jobs/relay (returns {processed}) or npm run job -- relay. Subscribers are idempotent (upserts, recompute) except that policy.issued → confirmation-and-cross-sell and renewal.link_requested → send-link send customer messages again if replayed after a partial success. Check data.handled before re-queuing those types
Verify	dead_letter count stable; pending returns to < 100 within minutes; for replayed policy.issued, confirm one confirmation message per order
Follow-up	RCA for the failing handler; add a regression test

Queries for RB-06 (run on the read-only replica unless the step says otherwise; updates only under a change ticket):

```
-- Q1
SELECT id, data→'type' AS type, data→'attempts' AS attempts, data→'lastError' AS
last_error, data→'handled' AS handled, occurred_at FROM domain_events WHERE status = 'dead_letter'
ORDER BY occurred_at DESC LIMIT 50;

-- Q2
SELECT count(*) FROM domain_events WHERE status = 'processing' AND updated_at < now() -
interval '10 minutes';

-- Q3: stuck processing → pending (normally unnecessary: auto re-claim after 5 min)
UPDATE domain_events SET status='pending', data=jsonb_set(data,{status},'pending'),
version=version+1, updated_at=now() WHERE status='processing' AND updated_at < now() - interval '10
minutes';

-- Q4: selected dead letters → pending (attempts reset; already-succeeded handlers in
data.handled are skipped)
UPDATE domain_events SET status='pending',
data=jsonb_set(jsonb_set(data,{status},'pending'),{attempts},'0'), version=version+1,
updated_at=now() WHERE status='dead_letter' AND id = ANY(ARRAY['<uuid1>','<uuid2>']::text[]);
```

RB-07 — Journey run skipped many touchpoints

Detect	ALR-33 JourneySkipSpike; log journey run complete with high skipped or deferred; audit journeys.run details; GET /api/touchpoints?status=skipped
Severity	Sev 2 (lost renewals); Sev 1 if the run was outside the contact window but marketing still went out (should be impossible)
Diagnose	Group the skip reasons with query Q1 below. Typical causes: (a) daily/weekly contact cap reached: overlapping journeys and triggers. (b) no marketing consent or not reachable on zalo_zns: data or consent issue (expected). (c) zalo_zns: failed ...: provider outage (RB-04). Touchpoints blocked only by the contact window are not skipped: they stay scheduled and appear as deferred in the run summary (KI-05 fixed). A high deferred count means the run executed outside 08:00–20:00 ICT: check that the CronJob schedule is in UTC (01:15 UTC = 08:15 ICT). Each run pages through all due touchpoints (no 5,000 cap)
Remediate	(c): touchpoints skipped only for transient reasons may be re-queued (change ticket, L3) with query Q2 below. Then re-run inside the window : POST /api/journeys/run {}. Production always uses the real clock: at and date are ignored unless DEMO_MODE=true (KI-33 fixed; the same applies to POST /api/voice/campaign). Deferred touchpoints execute automatically in the next in-window run. (a) and (b) are expected behaviour; review caps with Compliance if volumes look wrong
Verify	Re-run summary done increases; no marketing message with sentAt outside 01:00–13:00 UTC (query Q3 below returns 0)

Queries for RB-07 (run on the read-only replica unless the step says otherwise; updates only under a change ticket):

```
-- Q1
SELECT data→'result'→'reason' AS reason, count(*) FROM touchpoints WHERE status='skipped'
AND (data→'executedAt')::timestampz > now() - interval '1 day' GROUP BY 1 ORDER BY 2 DESC LIMIT
20;

-- Q2
UPDATE touchpoints SET status='scheduled', channel=NULL, data=jsonb_set((data - 'result' -
'executedAt') || '{"channel":null}', '{status}', '"scheduled"'), version=version+1, updated_at=now()
WHERE status='skipped' AND due_date ≥ CURRENT_DATE - 2 AND data→'result'→'reason' NOT SIMILAR TO
'%(consent|do-not-contact|cap reached|not reachable)%';

-- Q3
SELECT count(*) FROM messages WHERE (data→'marketing')::boolean AND (extract(hour from
sent_at AT TIME ZONE 'Asia/Ho_Chi_Minh') < 8 OR extract(hour from sent_at AT TIME ZONE
'Asia/Ho_Chi_Minh') ≥ 20);
```

RB-08 — Audit chain verification fails

Detect	ALR-40 (synthetic hourly GET /api/audit/verify → ok:false); the governance dashboard shows the chain broken
Severity	Sev 1, treated as a security incident until proven otherwise. Notify the TASCO CISO and the DPO
Diagnose	0. audit_log is protected by DB triggers that reject UPDATE, DELETE (trg_audit_log_immutable) and TRUNCATE (trg_audit_log_no_truncate), so a break implies someone with owner or superuser rights disabled a trigger, or a restore went wrong. Check SELECT tgname, tgenabled FROM pg_trigger WHERE tgrelid = 'audit_log'::regclass; (0 = enabled). 1. The result has {ok:false, brokenAt:<index>, reason:"prev hash mismatch" "hash mismatch"}. brokenAt is the 0-based position in ORDER BY seq. 2. Read the rows around brokenAt (query Q1 below). 3. hash mismatch → that row's content (details, actor, action...) changed after writing. prev hash mismatch → a row was deleted or inserted out of band, or a partial restore happened. 4. Compare with the latest backup/PITR copy of audit_log. 5. Check DB audit logs (pgaudit or provider logs) for direct UPDATE/DELETE on audit_log and who did it

Remediate	Do not "repair" the chain. Preserve evidence: snapshot the DB, export <code>audit_log</code> to WORM storage. Revoke any compromised DB credentials. Follow the security incident procedure (containment, legal assessment of notification duties under the PDP rules). Under the CISO's direction, the original rows may be restored from backup into a forensic copy
Verify	A forensic report identifies the affected range; <code>verify</code> passes on the restored forensic copy
Prevent	Keep the immutability triggers enabled; <code>audit_log</code> INSERT/SELECT -only for the app role (no UPDATE/DELETE/TRUNCATE grants, no table ownership); pgaudit enabled (logs ALTER TABLE ... DISABLE TRIGGER); nightly export of the chain head hash to an external store
Note	<code>verify</code> reads the whole table into memory (KI-12). At very large volumes run it on a read replica during off-peak hours

Queries for RB-08 (run on the read-only replica unless the step says otherwise; updates only under a change ticket):

```
-- Q1
SELECT seq, id, at, actor, action, entity_type, entity_id, prev_hash, hash FROM audit_log
ORDER BY seq OFFSET GREATEST(<brokenAt>-2,0) LIMIT 5;
```

RB-09 — Login lockouts / sign-in failure spikes

Detect	ALR-50 LoginFailureSpike (status="401" on POST /api/auth/login); ALR-51 lockouts (status="423"); ALR-52 login 429s; audit <code>auth.login_failed</code> , <code>auth.login_locked</code> , <code>auth.mfa_failed</code>
Diagnose	GET /api/audit?action=auth.login_failed&limit=200: look at <code>details.reason</code> (unknown_user, bad_password, inactive) and <code>details.ip</code> . Many usernames from few IPs → credential stuffing: Sev 2 security. One user, many failures → forgotten password. Many 429 from the office NAT IP at shift start → the per-IP login limit (10/min) is shared by agents behind one egress IP
Remediate	Lockout counts failures on both the password and the TOTP step. Credential stuffing: block the IPs at the WAF, confirm MFA is enforced for privileged roles, notify Security. Single user: the lock expires after 15 min (LOCKOUT_MINUTES); for urgent cases an admin unlocks the account with POST /api/users/:id/reset {"unlock":true} (SOP-08). Shift-start NAT: raise RATE_LIMIT_LOGIN_MAX through CAB, or make sure TRUST_PROXY=true, TRUST_PROXY_HOPS matches the number of proxies and the gateway passes the real client IP
Verify	401/423 rates back to baseline

RB-10 — Rule set activated by mistake → rollback

Detect	Business report (wrong scores, wrong template, journey volumes jump); log rule set activated; audit <code>rules.approved</code> ; GET /api/ops/status → <code>rules[]</code> version/checksum changed
Severity	Sev 1 if it causes customer harm (wrong price display, wrong wording, contacting the wrong audience); otherwise Sev 2
Remediate	1. Contain outbound contact immediately if the rule affects messaging or journeys: SOP-07 (suspend the journeys CronJob; do not launch voice campaigns). 2. Identify the last good version: GET /api/rules?kind=<kind> → the retired version with <code>supersededBy</code> = the bad id. 3. Rollback is maker-checker too : a <code>rule_author</code> calls POST /api/rules/<good-id>/rollback → creates a new draft <kind>@N+1 with the good payload → POST /api/rules/<kind>@N+1/submit → a different user with <code>rule_approver</code> (MFA) calls POST /api/rules/<kind>@N+1/approve {comment:"Incident INC-..."} . Restricted kinds (abac, copy_guard, contact_policy, commission, retention) must be approved by a <code>compliance_officer</code> . Keep the on-call author and approver pair reachable for emergency rollbacks. 4. All replicas pick up the new active version within ≤ 15 s (cache TTL; also invalidated by the <code>rules.activated</code> event). 5. If leads or journeys were affected: POST /api/leads/recompute (or <code>npm run job -- recompute</code>) to re-score and re-plan touchpoints

Verify	GET /api/ops/status shows the new version and checksum equal to the good one; spot-check POST /api/rules/simulate on 3 profiles; resume the journeys CronJob
Follow-up	Messages already sent cannot be recalled: Compliance assesses whether customer communication is needed

RB-11 — Payment captured but issuance failed (and other order mismatches)

Detect	Audit order.issuance_failed; orders with status issuance_failed_refunded or compensation_failed (ALR-84); nightly reconciliation result (GET /api/ops/jobs → reconciliation → mismatches: "compensation failed ... manual action", "payment failed — confirm with VETC wallet", "stuck in pending_payment > 1h", "missing payment reference", "policy ... missing"); customer complaint "charged, no certificate"
Severity	Sev 1 if more than a handful, or if any order is compensation_failed
Diagnose	List recent failed orders with query Q1 below. For each order: (1) issuance_failed_refunded: compensation completed (issued lines cancelled, full refund refunded); confirm the refund in the VETC wallet back-office. Partner orders show refund not_applicable (partner-collected premium). (2) compensation_failed: refund.status = refund_failed and/or cancel_failures lists policies that TASCO core did not cancel (query Q2 below) (3) payment_failed: check VETC for a capture anyway (timeout after capture, KI-18). (4) pending_payment > 1 h is flagged by reconciliation as "stuck"
Remediate	Decide with TASCO Finance per order: (a) Customer still wants cover : the quote was released to open; the customer retries in the app with a new Idempotency-Key (re-using the failed attempt's key returns 409 CONFLICT , KI-31 fixed); a new quote is needed after the 24 h quote expiry (service_levels.quoteTtlHours). (b) Refund failed : initiate the refund in the VETC back-office; record the refund ID in the ticket. (c) Cancellation failed : ask TASCO core operations to cancel the listed policies; record the decision. (d) Captured but payment_failed : refund via VETC, or complete issuance manually with TASCO core and inform the customer. Then run POST /api/ops/jobs/reconciliation
Verify	Reconciliation mismatches = 0 for the window; the daily settlement report agrees (finance)
Customer comms	VETC CS template "Giao dịch chưa hoàn tất – đã hoàn tiền" (Compliance-approved)

Queries for RB-11 (run on the read-only replica unless the step says otherwise; updates only under a change ticket):

```
-- Q1
SELECT id, status, data->'paymentRef' AS payment_ref, data->'policies' AS issued, data->'refund' AS refund, data->'cancelFailures' AS cancel_failures, data->'error' AS error, created_at
FROM orders WHERE status IN
('compensation_failed', 'issuance_failed_refunded', 'payment_failed', 'pending_payment') AND created_at
> now() - interval '2 days' ORDER BY created_at;

-- Q2
SELECT id, product, status FROM policies WHERE data->'orderId' = '<orderId>';
```

RB-12 — High latency

Detect	ALR-03 LatencySL0Breach (p95 per route); synthetics slow
---------------	--

Diagnose	1. Which routes? <code>histogram_quantile(0.95, sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))</code> . 2. Only routes that drain events inline (POST /api/*orders, POST /api/partner/v1/quotes, POST /api/data/ingest, POST /api/voice/sessions/:id/turns, PUT /api/customer/consent, POST /api/customer/expiry, POST /api/journeys/run, POST /api/ecosystem/events, POST /api/voice/campaign)? With <code>eventBacklog.pending</code> high, they are doing the global backlog's work (KI-04): drain via POST /api/ops/jobs/relay or scale out, and find out why the backlog built up (e.g. a full recompute triggered by a big ingest). 3. Orders only : integration latency (<code>integration_calls_total</code> retries; vendor status). 4. Everything : DB (CPU, locks, <code>pg_stat_statements</code> top), pod CPU throttling, GC/memory (RB-13). 5. Dashboards are heavy at scale (KI-12); limit their use or move them to a read replica. 6. Login slow at shift start: <code>scrypt</code> CPU; scale out
Remediate	Scale out (<code>kubectl scale</code> or raise the HPA minimum via CAB emergency change); kill long-running ad-hoc queries; postpone <code>recompute</code> and <code>ingest</code> batches to off-peak hours
Verify	p95 back under SLO for 30 min

RB-13 — Memory growth / suspected leak

Detect	ALR-60 <code>MemoryHigh</code> (<code>process_resident_memory_bytes > 80 %</code> of the container limit); OOMKilled restarts (ALR-61, <code>resets(process_uptime_seconds[1h])</code>)
Diagnose	1. Check the store first : <code>/health/ready</code> → <code>store</code> . If memory, the pod is holding all data in RAM: Sev 1 misconfiguration (production refuses to start without <code>DATABASE_URL</code> , so check <code>NODE_ENV</code> and <code>ALLOW_MEMORY_STORE</code>). 2. Growth correlated with traffic from many IPs → rate-limiter buckets (cleaned only above 50,000 keys). 3. Large requests: POST /api/data/ingest (5,000 records), dashboards and DSAR exports load large arrays. 4. GET /api/audit/verify loads the whole audit log (KI-12). 5. Take a heap snapshot on one pod: run with <code>NODE_OPTIONS=--heapsnapshot-signal=SIGUSR2</code> (set via CAB), <code>kill -USR2 1</code> in the container, copy the snapshot out, and hand it to L3
Remediate	Rolling restart as a temporary measure (graceful: readiness drops first); raise the memory limit through CAB; L3 fix
Verify	RSS flat over 6 h

RB-14 — Certificate verification failures

Detect	Complaints from customers, traffic police or inspection centres; GET /api/public/certificates/:certNo returning <code>valid:false</code> ; 429s on that route; synthetic check ALR-71
Diagnose	1. <code>{"valid":false,"reason":"not_found"}</code> → no policies row with that certificate number. Check the order (RB-11): issuance may have failed after TASCO core issued the policy. 2. <code>valid:false, state:"not_yet_in_force"</code> → expected for early renewals : cover starts the day after the current expiry; the page shows the start date; explain this to the caller. <code>state:"expired"</code> → the policy has ended. Any other state (e.g. <code>cancelled</code>) is the policy status. 3. Valid in the API but the page fails → front-end/static issue: <code>/verify/<certNo></code> serves <code>public/verify.html</code> ; check the release image and the CSP (<code>script-src 'self'</code>). 4. 429 → per-IP limits (each verification costs 2 tokens, so 150/min per IP); inspection centres behind one NAT IP can hit it: allow-list them at the gateway or raise the limit
Remediate	As per the cause; for a missing policy row after successful TASCO core issuance, L3 inserts the policy from the TASCO core record (data fix under change control)
Verify	The QR scan shows a valid certificate

RB-15 — Partner API failures

Detect	Partner reports; 401 spikes on <code>/api/partner/v1/*</code> ; <code>quotes_total{channel="partner_api"}</code> flat
Diagnose	401 → wrong or revoked key, or partner suspended (GET /api/partners); keys must start with <code>tpk_</code> . 404 "Quote not found" on bind → the quote belongs to another partner, or a typo. 422 → quote expired or converted. 409 on a retry → the <code>Idempotency-Key</code> belongs to a failed attempt (or the quote is being paid concurrently): retry with a new key. 403 → the key lacks the scope (<code>quote, purchase, policies:read</code>)

Remediate	Re-issue the key (SOP-04); unsuspend through the partner manager (PATCH /api/partners/:id {status:"active"})
-----------	--

4. Standard operating procedures

SOP-01 — Rotate the PII data-encryption key (DATA_KEYS)

Format: `DATA_KEYS="k1:<base64 32 bytes>,k2:<base64 32 bytes>"`. `DATA_KEY_ACTIVE` selects the key for new writes; it defaults to the **last** listed key. Ciphertext carries its key ID (`enc:v1:<keyId>:...`), so any key still listed can decrypt it.

Step	Action	Owner
0	CAB standard change; schedule off-peak (22:00 ICT)	L2
1	Generate the key in the vault/KMS: <code>openssl rand -base64 32</code> → store as k2. Never put it in git or tickets	Security
2	Phase 1: distribute without activating. Update the secret to <code>DATA_KEYS=k1:...,k2:...</code> and set <code>DATA_KEY_ACTIVE=k1</code> explicitly . Rolling restart of the Deployment and update of the CronJob/Job pods (they read the same secret)	L2
3	Verify every pod runs with the new secret (<code>kubectl rollout status</code> ; no unknown data key errors in logs)	L2
4	Phase 2: activate. Set <code>DATA_KEY_ACTIVE=k2</code> ; rolling restart. Skipping phase 1 lets old pods meet k2 ciphertext they cannot decrypt (unknown data key k2 → 500s)	L2
5	Verify new writes use k2: <code>SELECT count(*) FROM profiles WHERE data::text LIKE '%enc:v1:k2:%';</code> grows	DBA
6	Never remove k1 while any row still contains <code>enc:v1:k1:</code> . Rows are re-encrypted only when rewritten; there is no re-encryption job yet (KI-19)	Security
Rollback	Set <code>DATA_KEY_ACTIVE=k1</code> (keep k2 listed)	L2

`BLIND_INDEX_KEY` (phone search index) **must not be rotated** without a full rebuild of `profiles.phone_bidx` (no tooling, KI-19). Treat it as long-lived and protect it in the KMS.

SOP-02 — Rotate JWT_SECRET

Step	Action
Impact	There is a single HS256 secret with no key ID. Rotation immediately invalidates : all staff sessions (re-login + MFA), all customer sessions (1 h tokens), pending MFA challenges, and every signed renewal link already sent (the link HMAC key is derived from <code>JWT_SECRET</code> , KI-08; links otherwise expire after <code>LINK_TTL_DAYS</code> , default 30). Customers opening older reminders see "Link invalid or expired" and must open the app directly
Planned rotation	Every 90 days, or on staff turnover with secret access. Schedule after 20:00 ICT (no sends), ideally the evening before a journey run so that new links are generated the next morning. Announce to staff 24 h ahead
Steps	1. Generate ≥ 48 random bytes in the KMS. 2. Update the <code>JWT_SECRET</code> secret. 3. Roll out fast (<code>maxSurge 100 %</code>) to minimise the mixed period in which tokens signed by new pods fail on old pods. 4. Verify login works; <code>GET /api/auth/me 200</code>
Emergency (compromise)	Rotate immediately regardless of the time; Sev 1; disable affected users; review <code>auth.login</code> audit entries since the suspected exposure

SOP-03 — User offboarding

1. The HR/line - manager ticket triggers it (same day; within 1 h for involuntary exits).
2. Admin: `PATCH /api/users/<id> {"status":"disabled"}`. **Effective on the next request:** `authenticate()` reloads the user and rejects non-active users, so existing tokens stop working immediately.
3. Supervisor: reassign the user's open or claimed handoffs: `GET /api/handoffs?status=claimed` (filter `assignedTo`), then `PATCH /api/handoffs/<id> {"assignTo":"<new user>"}`.
4. If the user was a `rule_author`, review their drafts and pending rule sets (`GET /api/rules?status=pending_approval`) and reject or re-own them.
5. Remove VPN, IdP and console access. Rotate any shared secret the user knew (SOP-01/02/04).
6. Evidence: audit `user.updated`; attach it to the ticket. Quarterly access review: `GET /api/users`, signed by each line manager.

SOP-04 — Partner API key issue, rotation and revocation

Action	Procedure
Issue	Partner manager: <code>POST /api/partners/<partnerId>/keys</code> . The response contains <code>apiKey</code> (<code>tpk_...</code>) once ; only its SHA-256 is stored. Deliver it through a secure channel (split delivery: key by encrypted email, password by SMS). Record <code>keyId</code> and <code>prefix</code>
Rotate	Issue a new key → the partner deploys it → revoke the old key. Both are valid in between
Revoke	<code>DELETE /api/partners/keys/<keyId></code> → immediate (keys are looked up on every request). Unknown <code>keyId</code> ? There is no list endpoint; find it in the audit trail: <code>GET /api/audit?action=partner.api_key_issued&entityId=<partnerId></code> → <code>details.keyId</code> , <code>details.prefix</code>
Suspend a partner	<code>PATCH /api/partners/<id> {"status":"suspended"}</code> : all of the partner's keys stop working
Leak suspected	Revoke immediately (Sev 2 security); review <code>GET /api/partner/v1/policies</code> activity and orders by <code>partnerId</code> since the exposure

SOP-05 — Data subject requests (DSAR)

Step	Detail
Intake	Via the TASCO hotline/email or VETC CS → ITSM "DSAR" ticket. Verify identity per the TASCO procedure (no processing before verification). Legal clock per the PDP Law 2025 and Decree 13/2023/ND-CP (Legal confirms deadlines ; internal target: acknowledge within 1 business day, complete access within 72 h)
Locate	The data subject is identified by vehicle: profile id = canonical plate key (e.g. 30A12345). A staff user with <code>profile:read</code> can confirm it via <code>GET /api/customers/<id></code> (PII visible only with <code>profile:read_pii</code>)
Access / portability	<code>compliance_officer</code> (MFA): <code>POST /api/dsar/<id>/export</code> → JSON with profile, lead, policies, messages, source records, voice sessions, quotes, orders, claims and telesales tasks. Deliver as an encrypted archive. The customer can also self-serve via <code>GET /api/customer/data-export</code>
Erase	<code>POST /api/dsar/<id>/erase</code> . 422 "Active policy in force ..." → reply with the legal basis (insurance records retained until expiry plus the statutory period) and diarise erasure for after expiry. On success: name, phone and alt phones are nulled, the profile is DNC, the lead deleted, messages blanked, transcripts emptied and source PII nulled; future re-ingests do not re-identify it. Handoffs (name, masked phone, notes), claims (description, location) and voice-session signals are also cleared, and the subject's sessions are revoked (KI-29 fixed). Policies and orders are retained under the legal obligation, linked by plate only
Objection / withdraw consent	The customer uses the consent centre (<code>PUT /api/customer/consent</code>) or the voice bot ("đừng gọi nữa" → DNC). Staff cannot edit consent via API; via CS, record it in the ticket and have the customer use the app, or L3 applies it as a data fix

Step	Detail
Other systems	Forward the request to VETC (app, wallet) and TASCO core (policy admin), which have their own records
Close	Attach the audit entries <code>dsar.access_exported</code> / <code>dsar.erased</code> to the ticket

SOP-06 — Data correction by the data steward

Case	Procedure
Wrong policy expiry date	<code>data_steward (MFA): PATCH /api/customers/<id>/expiry {"expiryDate":"YYYY-MM-DD", "insurer":"...", "evidence":"<document reference>"}</code> . Evidence is mandatory and lands in the audit (<code>profile.expiry_corrected</code>); the lead is recomputed at once
DQ queue	<code>GET /api/dq/issues?type=<type> → fix at the source → POST /api/dq/issues/<id>/resolve {"resolution":"..."}</code> . If the next ingest detects it again, it reopens
Wrong name or phone	No direct edit API (data minimisation). Correct it in the authoritative source (VETC account, TASCO core) and let the next extract update it, or ingest the corrected record from that source: <code>POST /api/data/ingest {"source":"vetc_account", "records":[...]}</code> . Survivorship takes the higher-trust source
Plate mismatch / wrong person from the voice bot	DQ issue types <code>plate_mismatch</code> / <code>wrong_person</code> : confirm with VETC CS, then correct the phone in the source system
Bulk correction	Change ticket + CSV → ingest in batches of ≤ 2,000 records (KI-16); check the response's <code>rejected</code> and <code>profilesTouched</code>

SOP-07 — Emergency stop of outbound contact (kill switch)

There is no single application flag. Use these levers, from fastest:

- Suspend the journeys CronJob:** `kubectl -n tasco-growth patch cronjob tasco-growth-journeys -p '{"spec":{"suspend":true}}'` (no scheduled reminders, calls or telesales tasks).
- Stop manual runs:** instruct campaign managers not to use `POST /api/journeys/run` or `POST /api/voice/campaign`; optionally remove `journeys:run` from roles via CAB.
- Block marketing in rules** (maker-checker, ~15 s to converge): a `compliance_officer` approves a `contact_policy` version with `maxMarketingContactsPerDay: 0` (`contact_policy` is a restricted kind) → every marketing contact is blocked by the cap. Service messages (confirmations, lapsed notice) still flow.
- Event triggers** (`POST /api/ecosystem/events`) still send service messages and `wallet_topup` marketing (subject to caps); to stop them, pause the VETC event feed at the source.

Resume in reverse order; re-queue the touchpoints skipped during the stop if appropriate (RB-07).

SOP-08 — Unlock a user, reset MFA or handle a forgotten password

Case	Procedure
Account locked (5 failed password or code attempts)	The lock expires after 15 min (<code>LOCKOUT_MINUTES</code>). If the user cannot wait, an admin (<code>users:manage</code>) calls <code>POST /api/users/<id>/reset {"unlock":true}</code> . Find the id with <code>GET /api/users</code>
Lost or replaced phone (authenticator)	Verify the requester's identity per the TASCO procedure (ticket). The admin calls <code>POST /api/users/<id>/reset {"resetMfa":true}</code> (combine with <code>"unlock":true</code> if needed). A new TOTP secret is generated, existing tokens are revoked and the user self-enrols at the next sign-in (QR code shown only to the user). The admin never sees the seed

Case	Procedure
Forgotten password	There is no password-reset API. Once staff sign-in is federated to the TASCO IdP (ADR-007), the IdP handles it. Until then, under a change ticket approved by the TASCO IT security lead, L3 runs a one-off script that sets a temporary password with <code>mustChangePassword:true</code> ; the user must change it at the next sign-in, which also revokes earlier tokens. Record the action in the ticket, because the script bypasses the application audit trail
Constraints	An admin cannot reset their own account or change their own roles or status: ask another administrator. Every reset is audited as <code>user.reset</code> with the <code>unlock</code> and <code>resetMfa</code> flags. Codes cannot be reused (TOTP replay protection)
Hosted UAT only	Demo accounts are unlocked and reset automatically at every start-up when <code>DEMO_ACCOUNT_SYNC=true</code> ; MFA demo users use the published <code>DEMO_TOTP_SEED</code> keys or "Sign in with demo code (UAT)". Restarting the UAT service is therefore the quickest way to restore a locked demo account

SOP-09 — Re-running scheduled jobs

Job	How	Safe to re-run?
relay	<code>POST /api/ops/jobs/relay</code> or <code>npm run job -- relay</code>	Yes
reconcile	<code>POST /api/ops/jobs/reconciliation</code>	Yes (read-only)
retention	<code>POST /api/ops/jobs/retention</code>	Yes (idempotent deletes by cutoff)
journeys	<code>npm run job -- journeys</code> / <code>POST /api/journeys/run {}</code>	Yes. Already-executed touchpoints are not re-sent; outside 08:00–20:00 ICT marketing touchpoints are only deferred, so run it inside the window
recompute	<code>npm run job -- recompute</code> / <code>POST /api/leads/recompute</code>	Yes; heavy (hours at 6 M, KI-12): off-peak only
migrate	Migration Job before rollout	Yes (applied migrations are skipped)
rules	<code>npm run job -- rules</code>	Yes: only seeds kinds that have no version yet
seed	Refuses to run unless <code>DEMO_MODE=true</code> ; never used in production	—

5. Health-check routine (daily, L2)

Time (ICT)	Check	Pass
08:00	<code>GET /api/ops/status</code> : all integrations closed; <code>eventBacklog.dead_letter</code> unchanged; <code>rules[]</code> checksums match the change log	✓
08:00	Last night's reconciliation, retention: <code>GET /api/ops/jobs</code> → succeeded, mismatches = 0 (any <code>compensation_failed</code> or <code>payment_failed</code> order → RB-11)	✓
09:00	Journeys run at 08:15: <code>journey run complete</code> log; deferred close to 0; skip reasons normal; relay CronJob succeeding every 5 min	✓
09:00	Audit chain: synthetic verify ok	✓
12:00 / 17:00	Business KPIs: quotes and orders in line with the same weekday last week (Monitoring and Alerting §6)	✓
Weekly	Access review delta; secret ages; certificate expiries; backup restore test result (DR and BCP)	✓

6. L1 known - error quick answers (KEDB)

Symptom reported	Likely cause	L1 action
"Account temporarily locked"	5 failed sign-ins or codes	Wait 15 min; if urgent, ticket to the admin for an unlock (SOP-08)
First sign-in shows a QR code / must change password	Normal: self-enrolment of the authenticator and initial password change	Guide the user through enrolment (scan with an authenticator app)
Lost or replaced phone (authenticator)	MFA must be re-enrolled	Ticket to the admin → <code>POST /api/users/:id/reset</code> with <code>resetMfa</code> (SOP-08)
"Invalid code" although the code looks right	The code was already used, or the phone clock is wrong	Wait for the next code; check automatic time on the phone
"Too many sign-in attempts — wait a minute"	Login rate limit per IP	Wait 60 s; if the whole office is affected → L2 (RB-09)
Customer: "Link invalid or expired"	Link from before a JWT rotation, or a mistyped link	Ask the customer to open the VETC app directly → Insurance
Customer: "Quote expired — please re-quote"	Quote older than 24 h (<code>service_levels.quoteTtlHours</code>)	Re-quote in the app
Customer cannot pay for physical damage cover	<code>MOTOR_PD</code> needs a recorded vehicle inspection first	Explain that a TASCO assessor will contact them; L2 checks that the inspection was recorded
Customer charged, no certificate	Issuance failure	Sev 2 ticket to L2 with time, plate and order reference (RB-11)
Agent asks how to take payment	Staff never take payment: quotes are sent to the customer's app (<code>POST /api/quotes/:id/send</code>)	Explain the assisted-sale flow; the customer opens the quote and pays in the VETC app
QR says not valid but the customer just renewed	Cover starts the day after the old policy ends	Explain the start date
Agent cannot open a customer	Region restriction (ABAC)	Expected; the supervisor reassigns
Error with "quote the request id"	Unexpected 500	Ticket to L2 with the <code>requestId</code>

7. Incident communication

Audience	Channel	Owner	When
War room	Teams/Zalo "TASCO-GP-INCIDENT" bridge	Incident Commander	Sev 1/2 start
TASCO IT Head, Business Owner	Phone + email	Communications Lead	Sev 1 within 30 min; then hourly
VETC IT / CS	Shared incident channel	Communications Lead	Any customer-facing impact
Customers	In-app banner / ZNS service message (Compliance-approved templates)	VETC CS + TASCO Marketing	Sev 1 > 30 min on the purchase path

Audience	Channel	Owner	When
Partners	Partner status email	Partner Manager	Partner API impact > 15 min
Regulator / data subjects	Per Legal / DPO	DPO	Personal data breach (statutory timelines)

Post-incident: blameless RCA within 5 business days for Sev 1/2, with actions tracked to closure and runbook updates.

8. Escalation contacts

Named contacts are entered at G3 from the interface agreements (IIAs) and vendor contracts, and verified quarterly.

Party	Role	Contact source	Hours
TASCO IT Production Support	L2 on-call	Rota tool	24×7
iorta TechNXT	L3 engineering on-call	Rota tool	24×7 in hypercare
VETC IT	Wallet / SSO / app / events	VETC IIA	24×7
TASCO core IT	Policy admin	TASCO core IIA	Business hours + Sev 1 on-call
Zalo	OA / ZNS account manager	Zalo OA agreement	Business hours
SMS brandname provider	NOC	Provider contract	24×7
Voice - AI vendor	NOC	Vendor contract	24×7 during campaign hours
Cloud / DB provider	Support	Hosting contract	24×7
TASCO CISO / DPO	Security and privacy incidents	TASCO security directory	24×7 for Sev 1